

# Spiegazione LiveDataset

## 1 Obiettivo del progetto

Il progetto contenuto nel notebook `liverDataSet(1).ipynb` affronta un problema di *machine learning supervisionato* di tipo *classificazione binaria*.

L'obiettivo è costruire un modello capace di prevedere se un paziente appartiene a una delle due classi:

- Liver Disease
- No Liver Disease

cioè se il paziente presenta oppure no una malattia epatica, sulla base di alcune variabili cliniche come età, sesso, valori di bilirubina, enzimi epatici e proteine nel sangue.

In termini di *machine learning*, abbiamo:

Input = dati clinici del paziente

Output = classe del paziente

La domanda principale del progetto è:

dati certi valori medici, è possibile addestrare un modello che distingua in modo abbastanza affidabile i pazienti con malattia epatica da quelli senza malattia epatica?

## 2 Dataset utilizzato

Il dataset viene scaricato da Kaggle tramite `kagglehub`:

```
import kagglehub

path = kagglehub.dataset_download(
    "shauryasrivastava01/liver-patient-dataset"
)
```

Successivamente viene letto il file CSV:

```
path = "Datasets/liver_patient_dataset.csv"
df = pd.read_csv(path)
```

Il dataset contiene **583 righe** e **11 colonne**. Ogni riga rappresenta un paziente.

Tabella 1: Descrizione delle colonne del dataset

| Colonna   | Significato                 |
|-----------|-----------------------------|
| Age       | età del paziente            |
| Gender    | sesso del paziente          |
| TB        | Total Bilirubin             |
| DB        | Direct Bilirubin            |
| Alkphos   | Alkaline Phosphatase        |
| Sgpt      | enzima epatico SGPT / ALT   |
| Sgot      | enzima epatico SGOT / AST   |
| TP        | Total Proteins              |
| ALB       | Albumin                     |
| A/G Ratio | rapporto albumina/globulina |
| Selector  | variabile target            |

La variabile più importante per il progetto è **Selector**, perché è il *target*, cioè la classe da predire.

### 3 Variabile target: Selector

La colonna **Selector** contiene due valori testuali:

- Liver Disease
- No Liver Disease

Nel notebook viene visualizzata la distribuzione delle classi:

```
conteggi = df["Selector"].value_counts()

plt.bar(conteggi.index, conteggi.values)
plt.xlabel("Selector")
plt.ylabel("Frequenza")
plt.title('Distribuzione di "Selector"')
plt.show()
```

Questa parte serve per capire se il dataset è *bilanciato* oppure *sbilanciato*.

Un dataset è **bilanciato** quando le classi hanno circa lo stesso numero di esempi. Esempio ideale:

Liver Disease = 300  
No Liver Disease = 300

Un dataset è **sbilanciato** quando una classe è molto più frequente dell'altra. In questo caso il dataset è sbilanciato: i pazienti con **Liver Disease** sono molti di più dei pazienti con **No Liver Disease**.

Questo è un punto fondamentale del progetto, perché uno sbilanciamento può ingannare il modello.

### 4 Perché lo sbilanciamento è un problema?

Supponiamo di avere questo dataset:

90 pazienti con malattia  
10 pazienti senza malattia

Un modello molto stupido potrebbe predire sempre:

**Liver Disease**

In quel caso avrebbe:

90% di accuracy

ma sarebbe un pessimo modello, perché non riconoscerebbe mai la classe minoritaria. Quindi **l'accuracy da sola non basta**.

Nel progetto, infatti, vengono usate anche metriche come:

- precision
- recall
- f1-score
- balanced accuracy
- confusion matrix

Queste metriche sono molto più informative quando il dataset è sbilanciato.

## 5 Separazione tra feature e target

Nel notebook vengono separate le variabili di input dalla variabile target:

```
feature_names = set(df.columns) - set(["Selector"])
target_name = "Selector"
categorical_features = set(["Gender"])
continuous_features = feature_names - set(["Gender"])
```

Qui si dice:

Tutte le colonne tranne **Selector** sono feature. **Selector** è il target. **Gender** è categorica. Le altre variabili sono continue/numeriche.

Quindi:

$$X = \text{df senza Selector}$$
$$y = \text{df["Selector"]}$$

Dove:

$$X = \text{dati usati per predire}$$
$$y = \text{valore da predire}$$

Tabella 2: Esempio di riga del dataset

| Age | Gender | TB  | DB  | Alkphos | Sgpt | Sgot | TP  | ALB | A/G Ratio | Selector      |
|-----|--------|-----|-----|---------|------|------|-----|-----|-----------|---------------|
| 65  | Female | 0.7 | 0.1 | 187     | 16   | 18   | 6.8 | 3.3 | 0.90      | Liver Disease |

Il modello vede:

$$\begin{aligned} \text{Age} &= 65 \\ \text{Gender} &= \text{Female} \\ \text{TB} &= 0.7 \\ \text{DB} &= 0.1 \\ &\vdots \end{aligned}$$

e deve predire:

$$\text{Selector} = \text{Liver Disease}$$

## 6 Codifica delle variabili categoriche

I modelli di scikit-learn non lavorano direttamente con stringhe come:

- Female
- Male
- Liver Disease
- No Liver Disease

Per questo nel notebook si crea una copia codificata del dataset:

```
df_encoded = df.copy()
df_encoded["Gender"] = df_encoded["Gender"].astype("category").cat.codes
df_encoded["Selector"] = df_encoded["Selector"].astype("category").cat.codes
```

Dopo questa trasformazione:

Female → 0  
Male → 1  
Liver Disease → 0  
No Liver Disease → 1

Questa cosa è molto importante. Nel notebook, quando si parla di `class_1`, si sta parlando di:

No Liver Disease

non di:

Liver Disease

Quindi se il codice ottimizza `f1_class_1`, sta cercando di migliorare il riconoscimento dei pazienti **senza** malattia epatica.

## 7 Divisione train/test

Il dataset viene diviso in *training set* e *test set*:

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    df_encoded[feature_names],
    df_encoded[target_name],
    test_size=0.3,
    random_state=42,
    stratify=y
)
```

La divisione è:

70% → training set  
30% → test set

Il *training set* serve per addestrare il modello. Il *test set* serve per valutare il modello su dati mai visti.

Il parametro più importante qui è:

```
stratify=y
```

Questo serve a mantenere nel train e nel test una distribuzione simile delle classi. Per esempio, se nel dataset completo circa il 70% dei pazienti è nella classe 0 e il 30% nella classe 1, anche il training set e il test set manterranno più o meno quelle proporzioni. Questo è utile soprattutto quando il dataset è sbilanciato.

## 8 Primo modello: Decision Tree senza vincoli

Il primo modello addestrato è un *albero decisionale*:

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(random_state=42)
model = model.fit(X_train, y_train)
```

Un albero decisionale è un modello che prende decisioni tramite una sequenza di domande. Esempio semplificato:

TB ≤ 1.2?  
↓ sì  
controlla Sgpt

$\downarrow \text{Sgpt} \leq 35$   
 $\downarrow$  sì  
 predici classe 1  
 $\downarrow$  no  
 predici classe 0  
 $\downarrow$  no  
 controlla DB  
 $\downarrow \text{DB} \leq 0.5$   
 $\downarrow$  sì  
 predici classe 0  
 $\downarrow$  no  
 predici classe 0

Ogni nodo interno dell'albero contiene una condizione. Ogni foglia contiene una predizione.

## 9 Come decide un albero decisionale?

Un albero decisionale cerca di dividere i dati in gruppi sempre più “puri”. Un gruppo è puro quando contiene quasi solo esempi della stessa classe.

Per misurare l'impurità, `DecisionTreeClassifier` usa di default l'*indice di Gini*. La formula è:

$$\text{Gini} = 1 - \sum_{i=1}^n p_i^2 \quad (1)$$

dove  $p_i$  è la proporzione della classe  $i$  nel nodo.

**Esempio 1.** *Supponiamo un nodo con:*

50 pazienti classe 0  
 50 pazienti classe 1

*Allora:*

$$\begin{aligned}
 p_0 &= 0.5 \\
 p_1 &= 0.5 \\
 \text{Gini} &= 1 - (0.5^2 + 0.5^2) \\
 &= 1 - (0.25 + 0.25) \\
 &= 0.5
 \end{aligned}$$

*Il nodo è molto impuro.*

**Esempio 2.** *Supponiamo invece:*

100 pazienti classe 0  
 0 pazienti classe 1

*Allora:*

$$\begin{aligned}
 p_0 &= 1 \\
 p_1 &= 0 \\
 \text{Gini} &= 1 - (1^2 + 0^2) \\
 &= 0
 \end{aligned}$$

*Il nodo è perfettamente puro.*

L'albero cerca gli *split* che riducono il più possibile l'impurità.

## 10 Valutazione del primo albero

Dopo l'addestramento, il modello viene valutato sul test set:

```
from sklearn.metrics import confusion_matrix

y_pred = model.predict(X_test)
cm = confusion_matrix(y_test, y_pred)

print("Confusion matrix:\n", cm)
print("\nAccuracy:", cm.diagonal().sum() / cm.sum())
```

La matrice di confusione ottenuta è:

$$\begin{bmatrix} 96 & 29 \\ 28 & 22 \end{bmatrix}$$

Questa matrice va letta così:

Tabella 3: Matrice di confusione del primo albero decisionale

|        | Predetto 0 | Predetto 1 |
|--------|------------|------------|
| Vero 0 | 96         | 29         |
| Vero 1 | 28         | 22         |

Dato che:

0 = Liver Disease

1 = No Liver Disease

abbiamo:

- 96 pazienti con Liver Disease classificati correttamente
- 29 pazienti con Liver Disease classificati erroneamente come No Liver Disease
- 28 pazienti con No Liver Disease classificati erroneamente come Liver Disease
- 22 pazienti con No Liver Disease classificati correttamente

L'*accuracy* è:

$$\text{accuracy} \approx 0.6743 \text{ (circa 67.4\%)}$$

Il risultato non è ottimo. Il modello riconosce abbastanza la classe maggioritaria, ma fatica con la classe minoritaria.

## 11 Accuracy, precision, recall e F1-score

### 11.1 Accuracy

L'*accuracy* misura quante predizioni sono corrette sul totale:

$$\text{accuracy} = \frac{\text{predizioni corrette}}{\text{predizioni totali}} \quad (2)$$

Nel primo modello:

$$\begin{aligned} \text{accuracy} &= \frac{96 + 22}{175} \\ &= \frac{118}{175} \\ &\approx 0.674 \end{aligned}$$

## 11.2 Precision

La *precision* risponde alla domanda:

tra tutti quelli che il modello ha predetto come classe positiva, quanti erano davvero positivi?

Per la classe 1:

$$\text{precision}_1 = \frac{\text{veri 1 predetti 1}}{\text{tutti i predetti 1}} \quad (3)$$

Nel primo modello:

$$\begin{aligned} \text{precision}_1 &= \frac{22}{29 + 22} \\ &= \frac{22}{51} \\ &\approx 0.43 \end{aligned}$$

## 11.3 Recall

La *recall* risponde alla domanda:

tra tutti quelli che erano davvero classe positiva, quanti ne ha trovati il modello?

Per la classe 1:

$$\text{recall}_1 = \frac{\text{veri 1 predetti 1}}{\text{tutti i veri 1}} \quad (4)$$

Nel primo modello:

$$\begin{aligned} \text{recall}_1 &= \frac{22}{28 + 22} \\ &= \frac{22}{50} \\ &= 0.44 \end{aligned}$$

## 11.4 F1-score

L'*F1-score* è una media armonica tra precision e recall:

$$F1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}} \quad (5)$$

È utile quando vogliamo un compromesso tra precision e recall.

## 12 Problema del primo albero: overfitting

Un albero decisionale senza vincoli può crescere troppo. Se cresce troppo, può imparare quasi a memoria il training set. Questo fenomeno si chiama *overfitting*.

Un modello in *overfitting*:

- va molto bene sul training set
- va male o non migliora sul test set

Nel notebook viene visualizzato l'albero con:

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(150, 100))
plot_tree(model, filled=True, ax=ax)
plt.plot()
```

La dimensione enorme della figura suggerisce che l'albero completo sia troppo grande e difficile da interpretare.

## 13 Miglioramento: limitare la profondità dell'albero

Per controllare l'*overfitting* si prova un albero con profondità massima:

```
prunedModel = DecisionTreeClassifier(  
    random_state=42,  
    max_depth=5,  
    min_samples_split=2  
)
```

Il parametro:

```
max_depth=5
```

significa:

l'albero può avere al massimo profondità 5

Questo rende il modello più semplice. Tuttavia il risultato ottenuto è:

$$\text{Confusion matrix: } \begin{bmatrix} 104 & 21 \\ 41 & 9 \end{bmatrix}$$
$$\text{Accuracy: } \approx 0.6457$$

Quindi l'*accuracy* scende a circa 64.6%. La classe 1 viene riconosciuta molto male:

solo 9 veri positivi su 50

Questo modello è troppo sbilanciato verso la classe 0.

## 14 Pruning tramite ccp\_alpha

Il notebook usa poi il parametro di complessità `ccp_alpha`:

```
path = model.cost_complexity_pruning_path(X_train, y_train)  
ccp_alphas = path.ccp_alphas  
print(ccp_alphas)
```

Il *pruning* serve a potare l'albero, cioè a eliminare rami poco utili. L'idea è:

un albero più grande può essere più preciso sul training set, ma può generalizzare peggio.

Il parametro `ccp_alpha` controlla quanto penalizziamo la complessità dell'albero. La funzione obiettivo diventa, in modo semplificato:

$$\text{errore dell'albero} + \alpha \cdot \text{complessità dell'albero} \quad (6)$$

Se  $\alpha$  è piccolo, l'albero può rimanere grande. Se  $\alpha$  è grande, l'albero viene potato di più.

## 15 Ricerca degli iperparametri

Nel notebook viene fatta una ricerca manuale su più valori di:

- `max_depth`
- `ccp_alpha`
- `class_weight`

Per esempio:

```
for depth in range(4, 8):  
    for complexity in ccp_alphas:  
        clf = DecisionTreeClassifier(  
            max_depth=depth,  
            ccp_alpha=complexity,  
            class_weight="balanced",  
            random_state=42  
        )
```



Questa parte è una forma di *grid search* manuale. Si provano molte configurazioni e si sceglie quella che ottiene il risultato migliore sul test set.

Il primo risultato migliore trovato è:

Miglior max\_depth: = 6  
Miglior ccp\_alpha: = 0.0  
Train accuracy: = 0.7745  
Test accuracy: = 0.6914

La matrice di confusione è:

$$\begin{bmatrix} 79 & 46 \\ 8 & 42 \end{bmatrix}$$

Rispetto al primo albero:

classe 1 riconosciuta molto meglio

Infatti:

veri classe 1 = 50  
classe 1 correttamente predetti = 42

La *recall* della classe 1 diventa:

$$\text{recall}_1 = \frac{42}{50} = 0.84$$

Quindi il modello trova l'84% dei casi della classe 1. Però aumenta il numero di falsi positivi:

46 pazienti classe 0 vengono predetti come classe 1

Questo è il classico compromesso:

aumento la recall della classe minoritaria, ma rischio più errori sulla classe maggioritaria.

## 16 Balanced accuracy

Nel notebook viene calcolata anche la *balanced accuracy*:

```
from sklearn.metrics import balanced_accuracy_score  
print("Balanced accuracy:", balanced_accuracy_score(y_test, y_pred))
```

La *balanced accuracy* è la media delle recall delle classi. Per due classi:

$$\text{balanced accuracy} = \frac{\text{recall classe 0} + \text{recall classe 1}}{2} \quad (7)$$

Con la matrice:

$$\begin{bmatrix} 79 & 46 \\ 8 & 42 \end{bmatrix}$$

abbiamo:

$$\begin{aligned} \text{recall classe 0} &= \frac{79}{79 + 46} = \frac{79}{125} = 0.632 \\ \text{recall classe 1} &= \frac{42}{8 + 42} = \frac{42}{50} = 0.84 \end{aligned}$$

Quindi:

$$\begin{aligned} \text{balanced accuracy} &= \frac{0.632 + 0.84}{2} \\ &= 0.736 \end{aligned}$$

Il notebook infatti ottiene:

Balanced accuracy: 0.736

Questa metrica è più adatta dell'*accuracy* semplice quando le classi sono sbilanciate.

## 17 Uso dei pesi di classe

Il progetto prova poi a dare più importanza alla classe minoritaria usando `class_weight`. Nel notebook viene definita una funzione:

```
def train_best_weighted_tree(...):
```

La funzione prova diversi pesi:

```
weights_to_try = [  
    {0: 1, 1: 1},  
    {0: 1, 1: 1.5},  
    {0: 1, 1: 2},  
    {0: 1, 1: 2.5},  
    {0: 1, 1: 3},  
    {0: 1, 1: 4},  
    {0: 1, 1: 4.5},  
    {0: 1, 1: 5},  
    {0: 1, 1: 7},  
    {0: 1, 1: 9},  
    {0: 1, 1: 10}  
]
```

Un peso:

```
{0: 1, 1: 2.5}
```

significa:

gli errori sulla classe 1 pesano 2.5 volte di più degli errori sulla classe 0

Questo spinge l'albero a prestare più attenzione alla classe 1.

## 18 Miglior albero pesato

La migliore configurazione trovata è:

```
max_depth: = 6  
ccp_alpha: = 0.0  
class_weight: = {0 : 1, 1 : 2.5}  
train_accuracy: = 0.7745  
test_accuracy: = 0.6914  
test_balanced_accuracy: = 0.736  
precision_class_1: = 0.4773  
recall_class_1: = 0.84  
f1_class_1: = 0.6087  
macro_f1: = 0.6770
```

Matrice di confusione:

$$\begin{bmatrix} 79 & 46 \\ 8 & 42 \end{bmatrix}$$

Il risultato è lo stesso già visto con `class_weight="balanced"`. Questo modello è migliore del primo albero semplice se ci interessa riconoscere bene la classe 1.

Però bisogna stare attenti: la classe 1 è **No Liver Disease**. Se l'obiettivo medico fosse riconoscere soprattutto i malati, probabilmente bisognerebbe invertire la codifica o scegliere come positiva la classe **Liver Disease**.

## 19 Ensemble di tre alberi con majority voting

Il notebook prova poi una tecnica di *ensemble*. Un *ensemble* combina più modelli invece di usarne uno solo. Qui vengono addestrati tre alberi decisionali e le loro predizioni vengono combinate tramite voto di maggioranza.

La funzione principale è:

```
def majority_vote(predictions):
```

L'idea è semplice. Supponiamo che tre modelli predicano:

modello 1  $\rightarrow$  1

modello 2  $\rightarrow$  0

modello 3  $\rightarrow$  1

Due modelli su tre hanno votato 1, quindi la predizione finale è:

1

Se invece:

modello 1  $\rightarrow$  0

modello 2  $\rightarrow$  0

modello 3  $\rightarrow$  1

la predizione finale è:

0

Nel codice:

```
if votes_for_1 >= 2:
    final_predictions.append(1)
else:
    final_predictions.append(0)
```

## 20 Addestramento dei tre alberi

La funzione:

```
def train_three_trees(...)
```

usa:

```
from sklearn.model_selection import StratifiedKFold
StratifiedKFold(n_splits=3, shuffle=True, random_state=random_state)
```

Quindi il training set viene diviso in 3 parti stratificate. Ogni albero viene addestrato su una parte diversa dei dati. Questo crea tre modelli leggermente diversi.

L'idea generale è:

un singolo albero può essere instabile; più alberi possono compensarsi tra loro.

Questo principio è alla base anche delle *Random Forest*.

## 21 Risultato dell'ensemble

La migliore configurazione trovata senza *undersampling* è:

```
max_depth: = 5
ccp_alpha: = 0.0
class_weight: = {0 : 1, 1 : 5}
test_accuracy: ≈ 0.6057
test_balanced_accuracy: ≈ 0.706
precision_class_1: ≈ 0.4159
recall_class_1: = 0.94
f1_class_1: ≈ 0.5767
```

Matrice di confusione:

$$\begin{bmatrix} 59 & 66 \\ 3 & 47 \end{bmatrix}$$

Interpretazione:

47 elementi della classe 1 su 50 vengono riconosciuti correttamente

La *recall* della classe 1 è altissima:

$$\text{recall}_1 = \frac{47}{50} = 0.94$$

Però il modello predice troppi esempi come classe 1. Infatti:

66 esempi della classe 0 vengono classificati erroneamente come classe 1

Quindi l'*ensemble* aumenta molto la sensibilità verso la classe 1, ma peggiora la *precision*.

## 22 Undersampling

Il notebook prova poi una tecnica diversa per gestire lo sbilanciamento: l'*undersampling*. L'*undersampling* consiste nel ridurre il numero di esempi della classe maggioritaria, in modo da avere un training set bilanciato.

La funzione è:

```
def train_test_split_with_undersampling(...)
```

La cosa corretta fatta nel notebook è questa:

prima si divide in train/test, poi si applica *undersampling* solo sul training set.

Questo è molto importante. Non bisogna fare *undersampling* prima dello split, perché si rischia di alterare artificialmente anche il test set. Il test set deve rimanere realistico.

## 23 Come funziona l'undersampling nel codice

Il codice fa:

```
class_counts = train_df[target_col].value_counts()
min_class_count = class_counts.min()
```

Trova la classe meno numerosa. Poi, per ogni classe, prende lo stesso numero di esempi:

```
class_df_under = class_df.sample(
    n=min_class_count,
    random_state=random_state
)
```

Alla fine il training set bilanciato contiene:

$$0 \rightarrow 120$$

$$1 \rightarrow 120$$

Quindi il modello viene addestrato su:

120 esempi classe 0

120 esempi classe 1

## 24 Ensemble dopo undersampling

Dopo l'*undersampling* si riesegue la ricerca dell'*ensemble*. Il risultato migliore è:

```
max_depth: = 5
ccp_alpha: = 0.0
class_weight: = {0 : 1, 1 : 1.5}
test_accuracy: ≈ 0.6914
test_balanced_accuracy: ≈ 0.7756
precision_class_1: ≈ 0.4639
recall_class_1: ≈ 0.9574
f1_class_1: ≈ 0.625
macro_f1: ≈ 0.6814
weighted_f1: ≈ 0.7076
```

Matrice di confusione:

$$\begin{bmatrix} 76 & 52 \\ 2 & 45 \end{bmatrix}$$

Questo è uno dei risultati più interessanti del notebook. La classe 1 viene riconosciuta quasi sempre:

$$\text{recall}_1 \approx \frac{45}{47} = 0.957$$

Quindi la *recall* della classe 1 è circa 95.7%. La *balanced accuracy* è:

$$0.7756$$

che è migliore rispetto ai modelli precedenti. Anche qui, però, la *precision* della classe 1 resta bassa:

$$0.4639$$

cioè molti pazienti vengono predetti come classe 1 anche quando non lo sono.

## 25 Random Forest

Il notebook prova infine una *Random Forest*. Una *Random Forest* è un insieme di molti alberi decisionali. Mentre prima si usavano tre alberi, qui si usano molti più alberi, per esempio:

```
n_estimators_to_try = [100, 200, 300, 500]
```

Ogni albero viene addestrato con una certa casualità. La previsione finale è ottenuta combinando le predizioni di tutti gli alberi. In classificazione, di solito, si usa il voto di maggioranza.

## 26 Perché Random Forest può funzionare meglio di un singolo albero?

Un singolo albero decisionale è molto sensibile ai dati. Piccoli cambiamenti nel training set possono produrre un albero molto diverso. La *Random Forest* riduce questo problema perché combina molti alberi.

L'idea è:

un albero singolo può sbagliare in modo instabile; molti alberi insieme producono una previsione più robusta.

In generale:

Decision Tree = modello semplice ma instabile

Random Forest = insieme di alberi, più stabile

## 27 Ricerca degli iperparametri della Random Forest

Nel notebook viene definita la funzione:

```
def search_best_random_forest(...)
```

La funzione prova combinazioni di:

- `n_estimators`
- `max_depth`
- `class_weight`

I valori provati sono:

```
n_estimators_to_try = [100, 200, 300, 500]
depths_to_try = [4, 5, 6, 7, 8, None]
weights_to_try = [
    {0: 1, 1: 1},
    {0: 1, 1: 1.5},
    {0: 1, 1: 2},
    {0: 1, 1: 3},
    {0: 1, 1: 5},
    "balanced"
]
```

La metrica usata per scegliere il modello migliore è:

```
scoring_metric="macro_f1"
```

Il *macro-F1* è la media degli F1-score delle classi, senza pesare di più la classe più frequente. È utile quando vogliamo valutare il modello in modo più equo su entrambe le classi.

## 28 Risultato della Random Forest

La migliore configurazione trovata è:

```
n_estimators: = 200
max_depth: = None
class_weight: = {0 : 1, 1 : 1}
train_accuracy: = 1.0
test_accuracy: ≈ 0.6857
test_balanced_accuracy: ≈ 0.7178
precision_class_1: ≈ 0.4512
recall_class_1: ≈ 0.7872
f1_class_1: ≈ 0.5736
macro_f1: ≈ 0.6624
```

Matrice di confusione:

$$\begin{bmatrix} 83 & 45 \\ 10 & 37 \end{bmatrix}$$

Interpretazione:

83 elementi classe 0 corretti

45 elementi classe 0 sbagliati

10 elementi classe 1 sbagliati

37 elementi classe 1 corretti

La *Random Forest* ottiene una *accuracy* simile agli altri modelli, ma non supera l'*ensemble* con *undersampling* sulla *balanced accuracy*. Inoltre:

$$\text{train\_accuracy} = 1.0$$

Questo indica possibile *overfitting*. Il modello impara perfettamente il training set, ma sul test set arriva solo a circa 68.6%.

## 29 Confronto tra i modelli principali

Tabella 4: Confronto tra i modelli principali

| Modello                     | Accuracy test | Balanced accuracy | Recall classe 1 | F1 classe 1 |        |
|-----------------------------|---------------|-------------------|-----------------|-------------|--------|
| Albero base                 | 0.674         | 0.604             | 0.44            | 0.44        | model  |
| Albero max_depth=5          | 0.646         | 0.486             | 0.18            | 0.23        | tropp  |
| Albero pesato               | 0.691         | 0.736             | 0.84            | 0.609       |        |
| Ensemble 3 alberi           | 0.606         | 0.706             | 0.94            | 0.577       | recall |
| Ensemble +<br>undersampling | 0.691         | 0.776             | 0.957           | 0.625       | migl   |
| Random Forest               | 0.686         | 0.718             | 0.787           | 0.574       | stabi  |

Il modello più interessante nel notebook è:

**Ensemble di tre alberi + undersampling**

perché ottiene:

$$\text{balanced accuracy} \approx 0.7756$$

$$\text{recall classe 1} \approx 0.9574$$

$$\text{f1 classe 1} \approx 0.625$$

È il modello che riconosce meglio la classe 1.

## 30 Feature importance con Mutual Information

Alla fine il notebook calcola l'importanza delle variabili usando la *mutual information*:

```
from sklearn.feature_selection import mutual_info_classif
```

La *mutual information* misura quanta informazione una variabile fornisce sul target. In modo intuitivo:

se conoscere una variabile aiuta molto a prevedere il target, allora quella variabile ha *mutual information* alta.

Il risultato ottenuto è:

Sgpt  $\rightarrow$  0.072161  
Sgot  $\rightarrow$  0.068853  
Age  $\rightarrow$  0.066943  
TB  $\rightarrow$  0.053878  
DB  $\rightarrow$  0.048560  
Alkphos  $\rightarrow$  0.028152  
ALB  $\rightarrow$  0.025456  
A/G Ratio  $\rightarrow$  0.018387  
Gender  $\rightarrow$  0.000000  
TP  $\rightarrow$  0.000000

Quindi le variabili più informative risultano:

- Sgpt
- Sgot
- Age
- TB
- DB

Questa cosa ha anche senso dal punto di vista clinico, perché SGPT, SGOT e bilirubina sono misure legate al funzionamento del fegato.

## 31 Attenzione a un piccolo errore nel codice della mutual information

Nel notebook c'è questa parte:

```
categoriche = ["gender"]
```

Però la colonna si chiama:

```
"Gender"
```

con la G maiuscola. Quindi sarebbe meglio scrivere:

```
categoriche = ["Gender"]
```

Versione corretta:

```
categoriche = ["Gender"]
discrete_mask = []
for col in X.columns:
    if col in categoriche:
        discrete_mask.append(True)
    else:
        discrete_mask.append(False)
```

Dato che **Gender** è binaria, l'errore non distrugge completamente l'analisi, ma è comunque meglio correggerlo.



## 32 Sporcamiento del dataset

Il notebook introduce anche una funzione per “sporcare” variabili categoriche:

```
def corrupt_rows_cat(df, col, frac=0.1, random_state=None):
```

L’idea è simulare rumore nei dati. Per esempio, se `frac=0.1`, si modifica casualmente il 10% delle righe di una certa colonna. Nel caso di una variabile categorica come `Gender`, il codice può cambiare alcuni valori:

$0 \rightarrow 1$

$1 \rightarrow 0$

Questo serve per studiare quanto il modello è robusto quando i dati contengono errori.

Esempio pratico:

```
df_sporco = corrupt_rows_cat(  
    df=df_encoded,  
    col="Gender",  
    frac=0.1,  
    random_state=42  
)
```

Questo crea una copia del dataset in cui il 10% dei valori della colonna `Gender` viene modificato.

## 33 Attenzione a un errore nella funzione `corrupt_rows_cat`

Nel codice c’è questa riga:

```
if random_state != None: random_state = None
```

Questa riga annulla il `random_state`. Probabilmente l’intenzione era rendere la funzione riproducibile, ma così facendo si ottiene l’effetto contrario.

Una versione più corretta sarebbe:

```
import numpy as np  
  
def corrupt_rows_cat(df, col, frac=0.1, random_state=None):  
    corrupt_df = df.copy()  
    rng = np.random.default_rng(random_state)  
    unique_values = corrupt_df[col].unique()  
  
    if len(unique_values) <= 1:  
        print(f"Attenzione: la colonna '{col}' ha un solo valore unico.")  
        return corrupt_df  
  
    n_rows_to_corrupt = int(frac * len(corrupt_df))  
    indices_to_corrupt = rng.choice(  
        corrupt_df.index,  
        size=n_rows_to_corrupt,  
        replace=False  
    )  
  
    for idx in indices_to_corrupt:  
        current_value = corrupt_df.at[idx, col]  
        pool_of_new_value = unique_values[unique_values != current_value]  
        corrupt_df.at[idx, col] = rng.choice(pool_of_new_value)  
  
    return corrupt_df
```

Così, usando lo stesso `random_state`, si ottiene sempre lo stesso dataset corrotto.

## 34 Osservazione importante: quale classe stiamo ottimizzando?

Questo è probabilmente il punto più importante da spiegare in una relazione.

Nel notebook:

0 = Liver Disease

1 = No Liver Disease

Però molte funzioni ottimizzano:

```
scoring_metric="f1_class_1"
```

Quindi stanno ottimizzando la classe:

No Liver Disease

Questo non è sbagliato in assoluto, ma va dichiarato. In un contesto medico, spesso ci interessa soprattutto non perdere i malati. In quel caso potrebbe essere più naturale codificare:

Liver Disease  $\rightarrow 1$

No Liver Disease  $\rightarrow 0$

Così la classe positiva sarebbe la malattia. Una versione più chiara sarebbe:

```
df_encoded["Selector"] = df["Selector"].map({
    "No_Liver_Disease": 0,
    "Liver_Disease": 1
})
```

In questo modo:

1 = paziente malato

0 = paziente non malato

E allora metriche come `recall_class_1` avrebbero un significato clinico più naturale:

quanti pazienti malati riesco a individuare?

## 35 Nota su `classification_report`

In alcune celle del notebook compare:

```
print(classification_report(y_pred, y_test))
```

L'ordine corretto normalmente è:

```
print(classification_report(y_test, y_pred))
```

La funzione si usa così:

```
classification_report(y_true, y_pred)
```

dove:

$y\_true$  = valori reali

$y\_pred$  = predizioni del modello

Quindi la forma corretta è:

```
print(classification_report(y_test, y_pred))
```

Nel notebook in alcune celle l'ordine viene poi usato correttamente.

## 36 Struttura generale del progetto

Possiamo riassumere il progetto come una *pipeline* di *machine learning*:

1. Importazione librerie
2. Download e lettura dataset
3. Esplorazione iniziale dei dati
4. Analisi della distribuzione del target

5. Codifica delle variabili categoriche
6. Train/test split
7. Addestramento di un Decision Tree base
8. Valutazione con confusion matrix e metriche
9. Miglioramento tramite max\_depth e pruning
10. Ricerca di iperparametri
11. Gestione dello sbilanciamento con class\_weight
12. Ensemble di tre alberi
13. Undersampling del training set
14. Random Forest
15. Analisi dell'importanza delle variabili
16. Simulazione di rumore nei dati

## 37 Conclusione

Il progetto mostra un percorso abbastanza completo di classificazione binaria su dati clinici.

Il punto di partenza è un semplice albero decisionale, che ottiene circa il 67% di *accuracy*, ma fatica a riconoscere bene la classe minoritaria.

Successivamente vengono introdotte tecniche più avanzate:

- pruning
- class\_weight
- balanced accuracy
- ricerca degli iperparametri
- ensemble
- undersampling
- random forest
- mutual information

Il risultato migliore, considerando la *balanced accuracy* e la capacità di riconoscere la classe 1, viene ottenuto con:

**ensemble di tre alberi + undersampling**

che raggiunge:

accuracy  $\approx 0.691$

balanced accuracy  $\approx 0.776$

recall classe 1  $\approx 0.957$

f1 classe 1  $\approx 0.625$

Tuttavia bisogna interpretare bene la codifica delle classi:

classe 0 = Liver Disease

classe 1 = No Liver Disease

Quindi il progetto, così com'è scritto, ottimizza spesso il riconoscimento della classe **No Liver Disease**. Per renderlo più coerente con un problema medico, sarebbe consigliabile ricodificare il target in modo che:

1 = Liver Disease

0 = No Liver Disease

e poi ottimizzare la *recall* o l'*F1* della classe malata.

In sintesi, il progetto è buono perché non si limita ad addestrare un singolo modello, ma esplora diversi problemi reali del *machine learning*:

- dataset sbilanciato
- overfitting
- scelta delle metriche
- interpretabilità
- ricerca degli iperparametri
- robustezza ai dati rumorosi

La parte più importante da valorizzare nella relazione è che l'***accuracy* da sola non basta**: in un problema clinico bisogna guardare attentamente *precision*, *recall*, *F1-score*, *balanced accuracy* e matrice di confusione.